Génie logiciel

UML (*Unified Modeling Language*) *ou langage unifié pour la modélisation objet*)

Chapitre 1: Introduction et cas d'utilisation

Plan de la présentation

- Introduction au génie logiciel
- Logiciel
- Crise du logiciel
- Processus de développement
- Approche systémique
- Modélisation
- Ce quoi UML ?
- Diagramme des cas d'utilisation

Génie logiciel

- **Ingénierie** (ou **génie**) : Ensemble des fonctions allant de la conception et des études à la responsabilité de la construction et au contrôle des équipements d'une installation technique ou industrielle. Exemple: Génie civil, naval, aéronautique, mécanique, chimique...
- **Génie logiciel** correspond à appliquer les méthodes classiques d'ingénierie au domaine du logiciel.

Génie logiciel

- **Définition** : Ensemble des méthodes, des techniques et des outils dédiés à la conception, au développement et à la maintenance des systèmes informatiques.
- **Objectif** : Avoir des procédures systématiques pour des logiciels de grande taille afin que :
  - la spécification corresponde aux besoins réels du client;
  - le logiciel respecte sa spécification;
  - les délais et les coûts alloués à la réalisation soient respectés.

Logiciel: définitions

- Ensemble de programmes qui permet à un système informatique d'assurer une tâche ou une fonction en particulier.

  Logiciel = programme + utilisation ;
- Ensemble des programmes, des règles, et des documentations relatif au fonctionnement de l'ensemble des traitements d'un SI.

Logiciel: caractéristiques

- Environnement : peut être composé des **utilisateurs** du logiciel (différents selon ses fonctionnalités), **d'autres logiciels** (en amont, ou en aval), ainsi que des **matériels** utilisés (ordinateur, serveurs, capteurs …etc.).
- Spécification: elle consiste à ce que doit faire le logiciel, ensemble de critères que doivent satisfaire son fonctionnement interne et ses interactions avec son environnement.

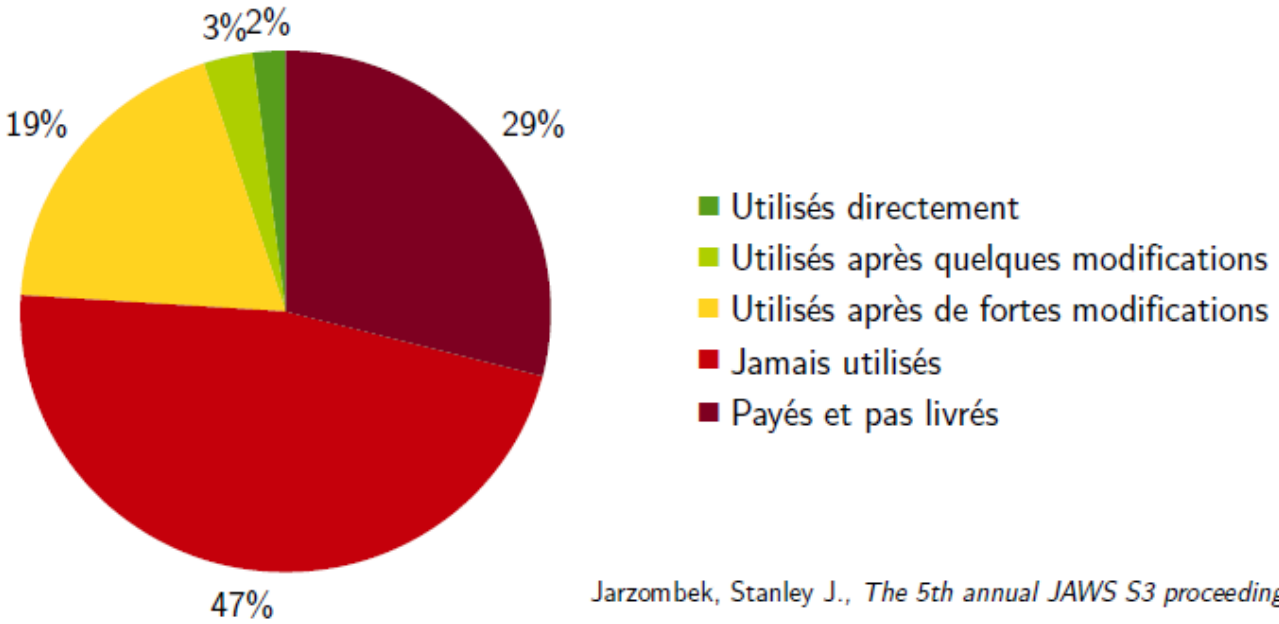
Crise du logiciel des années 60

- Constat du développement logiciel des années 60 :
  - délais de livraison non respectés
  - budgets non respectés
  - ne répond pas aux besoins de l'utilisateur ou du client
  - difficile à utiliser, maintenir, et faire évoluer

Exemple 1: crise du logiciel en 1995

Étude du DoD 1995

Étude du *Department of Defense* des États-Unis sur les logiciels produits dans le cadre de 9 gros projets militaires

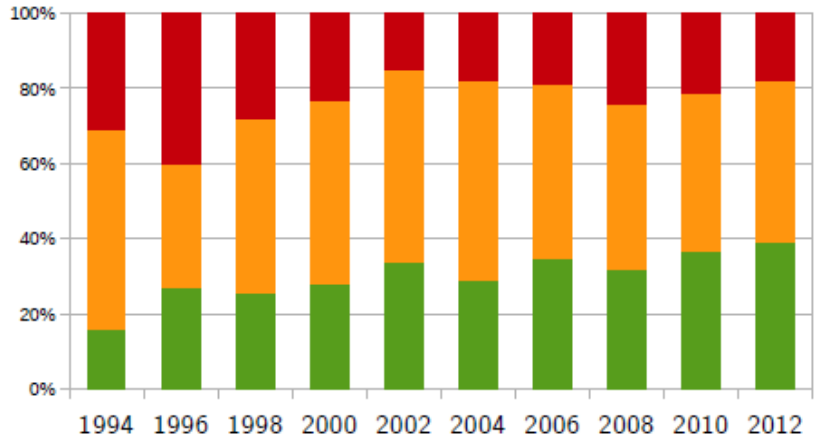


Jarzombek, Stanley J., *The 5th annual JAWS S3 proceedings*, 1999

Exemple 2: étude du Standish group

Étude du Standish group

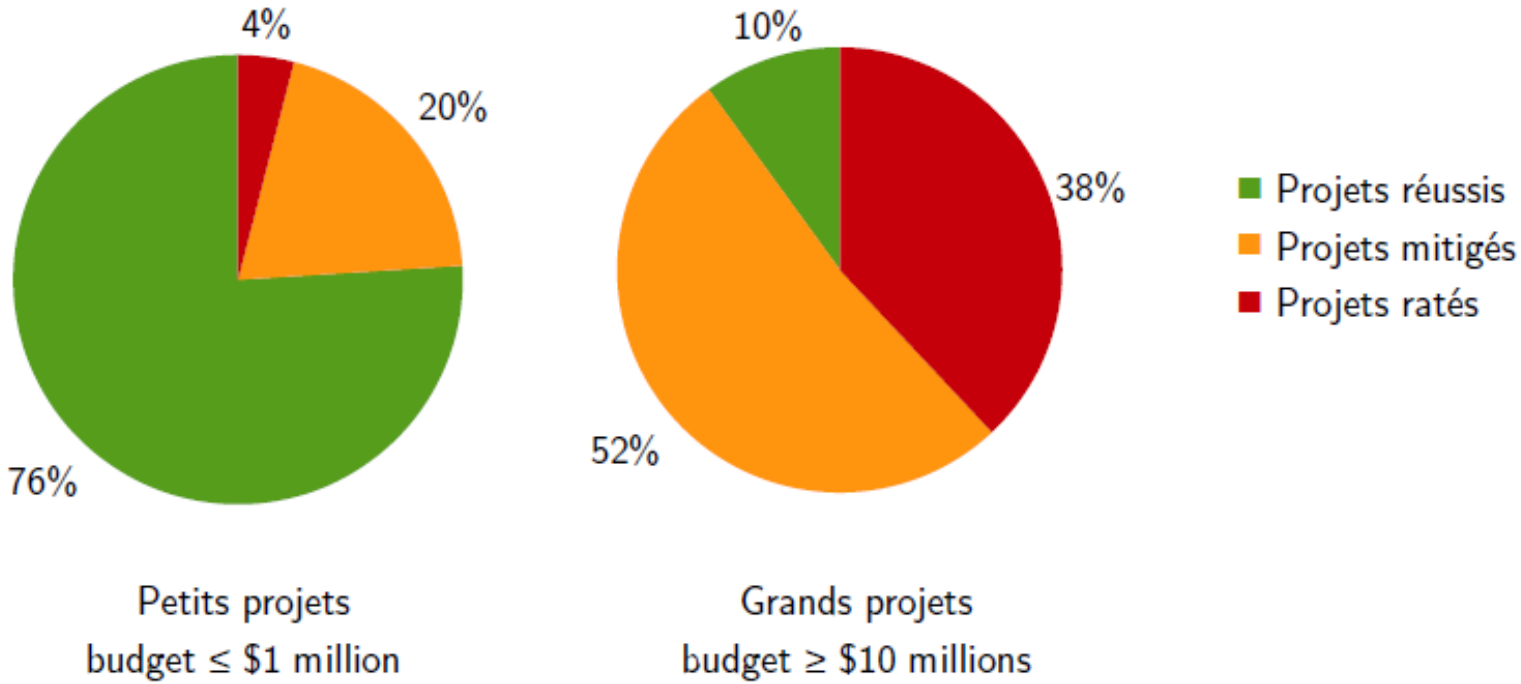
Enquête sur des milliers de projets, de toutes tailles et de tous secteurs



Standish group, *Chaos Manifesto 2013 - Think Big, Act Small*, 2013

- Projets réussis : achevés dans les délais et pour le budget impartis, avec toutes les fonctionnalités demandées
- Projets mitigés : achevés et opérationnels, mais livrés hors délais, hors budget ou sans toutes les fonctionnalités demandées
- Projets ratés : abandonnés avant la fin ou livrés mais jamais utilisés

Exemple 3: petits vs grands projet.



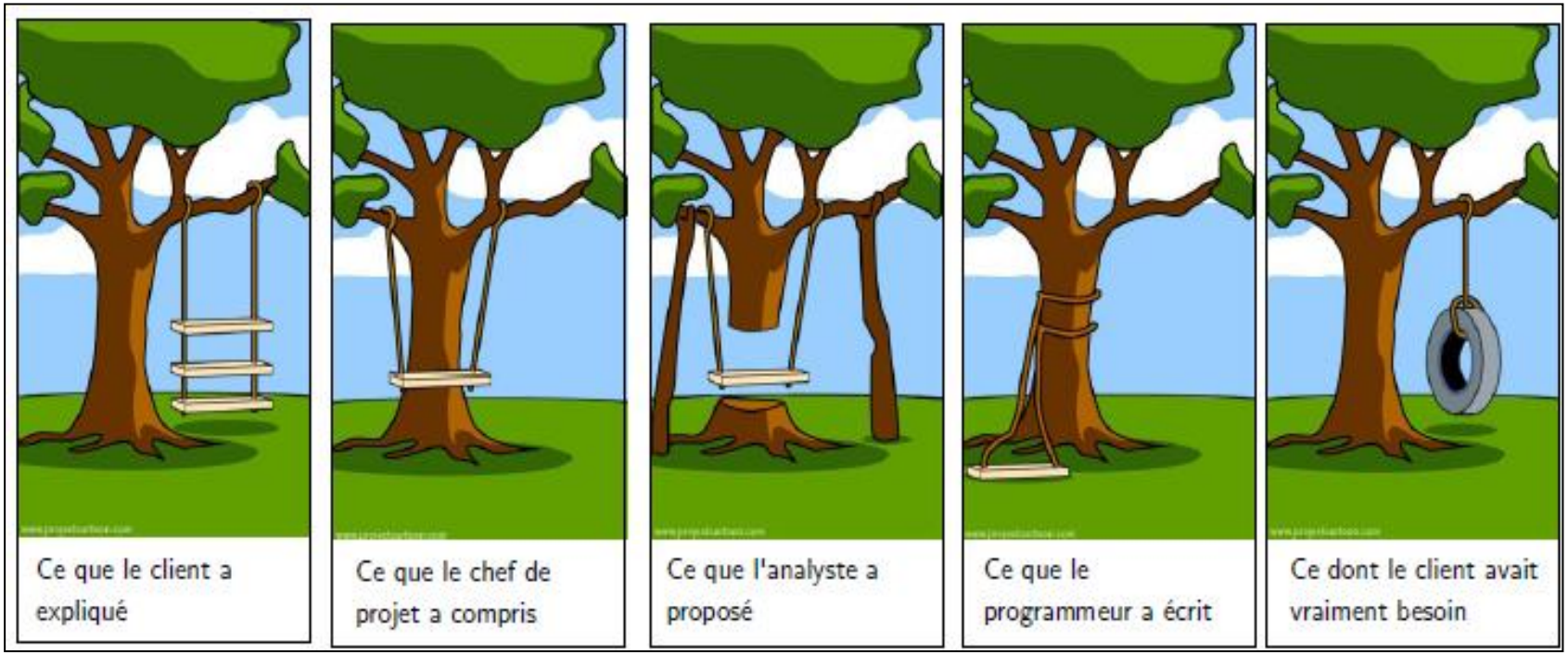
Standish group, *Chaos Manifesto 2013 - Think Big, Act Small*, 2013

Faible qualité du logiciel: raisons

- Mauvaise compréhension des besoins : on remarque qu'il y'a eu négligence de la phase d'analyse du besoin du client, mais aussi l'implication du client est absente.
- Absence des méthodes à utiliser : aucune méthode de conception n'a été utilisée, il y'a eu aussi un manque des outils de vérification/validation.
- Taille du projet plus ou moins complexe, alors que la taille de l'équipe plus ou moins réduite.

Faible qualité du logiciel: raisons

Mauvaise compréhension du besoin du client :



Critères de Qualité d'un logiciel

- **Fiabilité** (exactitude) : correspond aux besoins.
- **Robustesse** : tolérance aux pannes
- **Extensibilité** : évolution acceptée
- **Réutilisabilité** : possibilité de l'utiliser intégralement ou en partie dans d'autres projets.
- **Compatibilité** : possibilité de combiner son code avec d'autres codes.
- **Portabilité** : possibilité de l'exécuter sur différentes plates-formes.
- **Efficacité** : utilisation des ressources de la plus meilleure façon possible.

Processus de développement logiciel : activités

- Analyse des besoins
- Spécification du cahier des charges
- Conception
- Programmation
- Validation et vérification
- Livraison
- Maintenance

Pour chaque activité : Utilisation et production de documents

Analyse et spécification

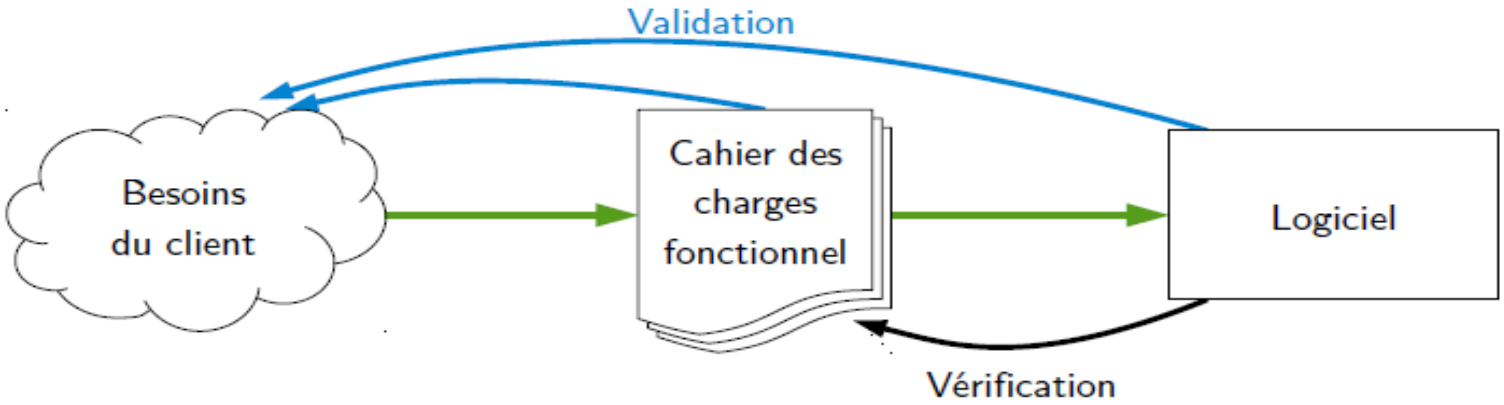
- Analyse des besoins : il s'agit de comprendre les besoins du client, à fin de fixer les objectifs généraux, l'environnement du futur système, les ressources disponibles ..etc.
- Spécification: il s'agit d'établir une description claire de ce que doit faire le logiciel pour clarifier le cahier des charges.

Conception et programmation

- Conception: consiste à élaborer une solution concrète réalisant la spécification du besoin :
  - Description architecturale sous forme des composants
  - Réalisation des fonctionnalités par les composants
  - Réalisation des exigences non fonctionnelles (performance, sécurité…etc.)
- Programmation : Implémentation de la solution conçue :
  - Choix de l'environnement de développement, du/des langage(s) de programmation, de normes de développement...etc.

Validation et vérification

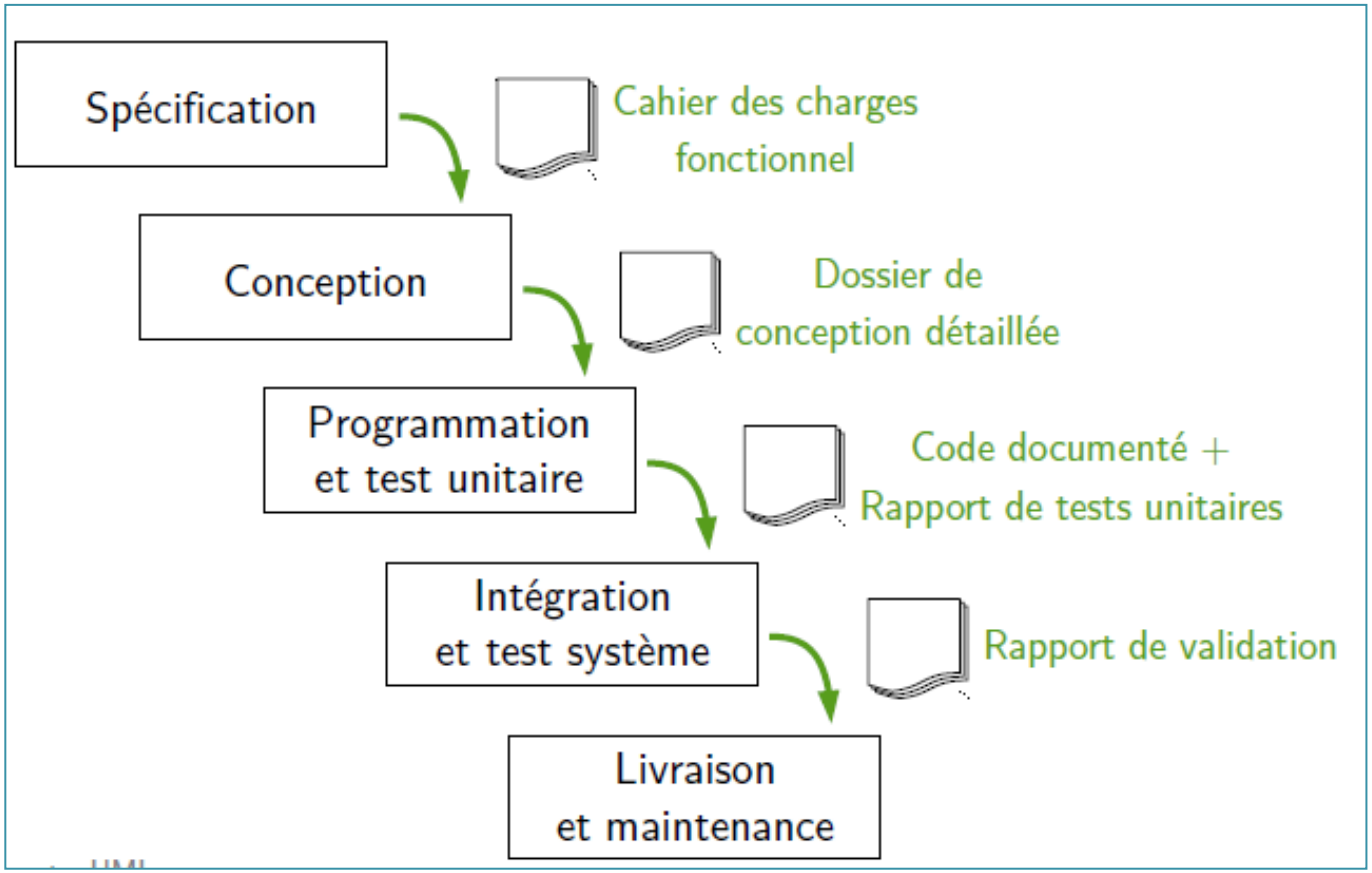
- Validation : assurer que les besoins du client sont satisfaits (au niveau de la spécification, du produit fini...).
- Vérification : assurer que le logiciel satisfait sa spécification.



Types de maintenance

- Correction : identifier et corriger des erreurs trouvées après la livraison;
- Adaptation : adapter le logiciel aux changements dans l'environnement (format des données, environnement d'exécution...etc.);
- Perfection : améliorer la performance, ajouter des fonctionnalités,..etc.

Processus en cascade

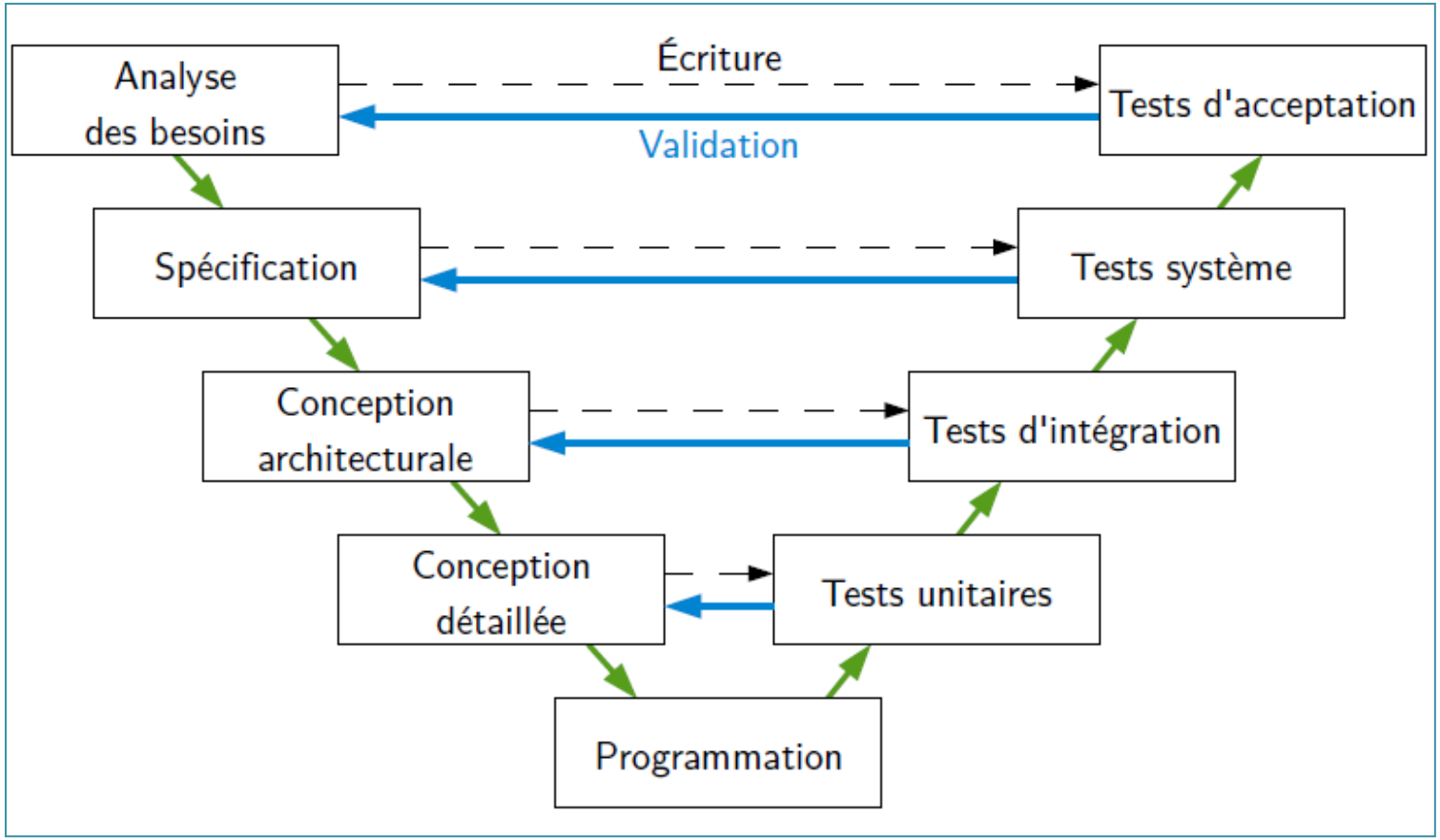


Limites du processus en cascade

- Découverte d'une erreur entraîne retour à la phase à l'origine de l'erreur et nouvelle cascade, avec de nouveaux documents...
- Coût de modification d'une erreur est important,

Pas toujours adapté à une production logicielle, en particulier si besoins du client changeants ou difficiles à spécifier.

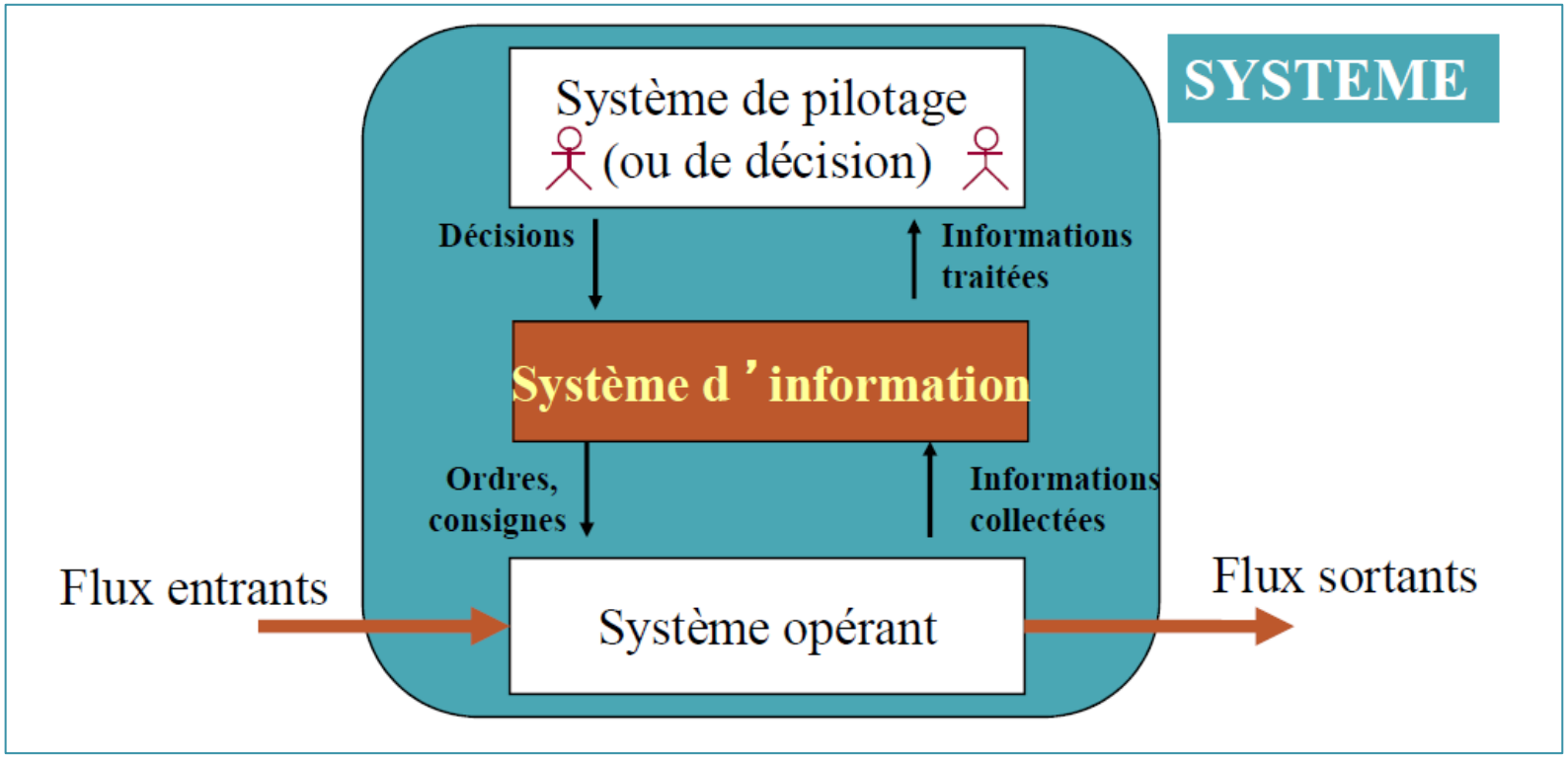
Processus de développement en V



Niveaux de tests du processus en V

- Test unitaire : test de chaque unité de programme (méthode, classe, composant), indépendamment du reste du système
- Test d'intégration : test des interactions entre composants (interfaces et composants compatibles)
- Test système : test du système complet par rapport à son cahier des charges
- Test d'acceptation (recette) : fait par le client, validation par rapport aux besoins initiaux.

Approche systémique



Quelques Exemples

Exemple 1 : Une compagnie de transport

- **Flux physiques** :
  - Les trains, les wagons, les voyageurs, les billets, les réservations, le paiement d'un billet, etc...
- **Flux d'information** :
  - des informations sur le trajet: Trajet ,ville départ, ville d'arrivée, heure de départ, heure d'arrivée, Numéro Billet, Numéro Réservation, etc...
  - des informations sur les CA : CA d'une ligne, etc...
- **Système de décision** :
  - Ouverture de nouveaux trajets
  - Définition de la grille tarifaire
  - Définition des trajets etc...

Exemple 2 : Un hôpital

- **Flux physiques** :
  - Les équipements médicaux et non médicaux, les personnels médicaux, les unités médicales, les services, ...
- **Flux d'information** :
  - des informations sur le patient: Identifiant Patient, Nom, Prénom, etc...
  - des informations sur les consultations : date de consultation, le médecin, le motif de la consultation
  - des informations sur les opérations : date l'opération, coût de l'opération, le chirurgien, l'acte chirurgicale, la durée, ...
- **Système de décision** :
  - Ouverture de nouvelles services de soins
  - Acquisition de nouveaux équipements médicaux
  - Définition de la grille tarifaire des services de soins
  - Planification des opérations...

La modélisation en génie logiciel

- Modèle : c'est la simplification de la réalité, consiste à produire une version abstraite de l' objet désiré.
- Objectifs de la modélisation : c'est mieux le comprendre (modélisation en physique) ou mieux le construire (modélisation en ingénierie).
- En génie logiciel : elle permet d'aider à la réalisation d'un logiciel à partir des besoins du client,
  - Modélisation = spécification + conception.

C'est quoi UML

- UML (*Unified Modeling Language) est un langage de modélisation orientée objet standardisée par l'OMG.*
- Elle est héritée de plusieurs autres méthodes telles que OMT (*Object Modeling Technique) et OOSE (Object Oriented Software Engineering) et Booch. Les principaux auteurs* de la notation UML sont Grady Booch, Ivar Jacobson et Jim Rumbaugh.

Quelques avantages d'UML

- Besoin de modéliser pour construire un logiciel
  - Modélisation des aspects statiques et dynamiques
  - Modélisation à différents niveaux d'abstraction et selon plusieurs vues
  - Indépendant du processus de développement
- Besoin de langages normalisés pour la modélisation
  - Langage semi-formel
  - Standard très utilisé
- Conception orientée objet
  - Façon efficace de penser le logiciel
  - Indépendance du langage de programmation

Conception OO avec UML

- Principes
  - Concept du domaine d'application = objet décrit par état (attributs) + comportement (opérations)
  - Liens entre concepts : héritage, agrégation, composition...
- Caractéristiques des objets
  - Identité : objet = entité unique (mêmes attributs $\not\Rightarrow$ même objet)
  - Classification (classe) : regroupement des objets de même nature (attributs + opérations)
  - Polymorphisme : comportement différent d'une même opération dans différentes classes
  - Héritage : partage hiérarchique des attributs et opérations

Les diagrammes utilisés avec UML

UML définit neuf diagrammes : cinq pour les aspects statiques et quatre pour les aspects dynamiques:

Aspects statiques :	***Aspects dynamiques :***
➢ *Les diagrammes de classe* ➢ *Les diagrammes d'objets* ➢ *Les diagrammes de cas d'utilisation* ➢ *Les diagrammes de composants* ➢ *Les diagrammes de déploiement*	➢ *Les diagrammes de séquence* ➢ *Les diagrammes de collaboration* ➢ *Les diagrammes d'états-transition* ➢ *Les diagrammes d'activités*

Diagramme de cas d'utilisation

CAS D'UTILISATION : les besoins

Une application doit rendre des services à ses utilisateurs en répondant à leurs besoins.

La première Phase de Développement d'une application consiste donc à identifier ces besoins et à les formaliser afin de pouvoir les traiter sans ambiguïtés.

Les besoins formalisés serviront à déterminer les fonctionnalités à développer et à élaborer une interface homme-machine d'accès à ces fonctionnalités.

Définition :Un besoin est une action fonctionnelle possible d'un processus.

À partir d'une situation de gestion donnée, nous allons tenter de décrire les processus existants dans ce système d'un point de vue fonctionnel.

CAS D'UTILISATION : concepts

Les besoins répondent à des exigences :

- Fonctionnelles : liées à l'utilisation du système
- Non fonctionnelles : liées à la qualité, au performance du système.

Concepts	Représentation
Le système	
Les cas d'utilisation	
Relation	
Les acteurs	

Définition des concepts

- **Le système** peut être une application informatique. C'est un ensemble de **cas d'utilisation.**

- **Acteur** : entité externe qui agit sur le système et :
  - prend les décisions contrairement à un élément logiciel
  - possède un rôle par rapport au système
  - soit un utilisateur
  - soit un autre système

Définition des concepts (2)

- **Cas d'utilisation** est un ensemble des actions réalisées par le système en réponse à une action d'un acteur :
  - suite d'interactions entre un acteur et le système
  - correspond à une fonction visible par l'utilisateur
  - permet d'atteindre un objectif aux yeux de l'utilisateur
  - doit être utile
- Le diagramme des cas d'utilisation est la représentation graphique de l'expression des fonctionnalités recherchées pour le système et des interactions qui existent entre ses acteurs.

Cette représentation comporte trois éléments : le ou les acteurs(s), le ou les cas d'utilisation et le ou les relation(s).

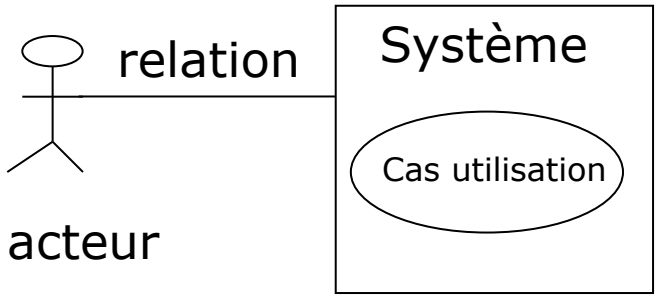
Remarque sur les acteurs

Ne pas confondre acteur et personne utilisant le système :

- une même personne peut jouer plusieurs rôles
- plusieurs personnes peuvent jouer un même rôle
- un acteur n'est pas forcément une personne physique…

Il peut exister plusieurs types d'acteurs :

- Utilisateur principaux
- Utilisateurs secondaires
- Périphériques externes
- Systèmes externes



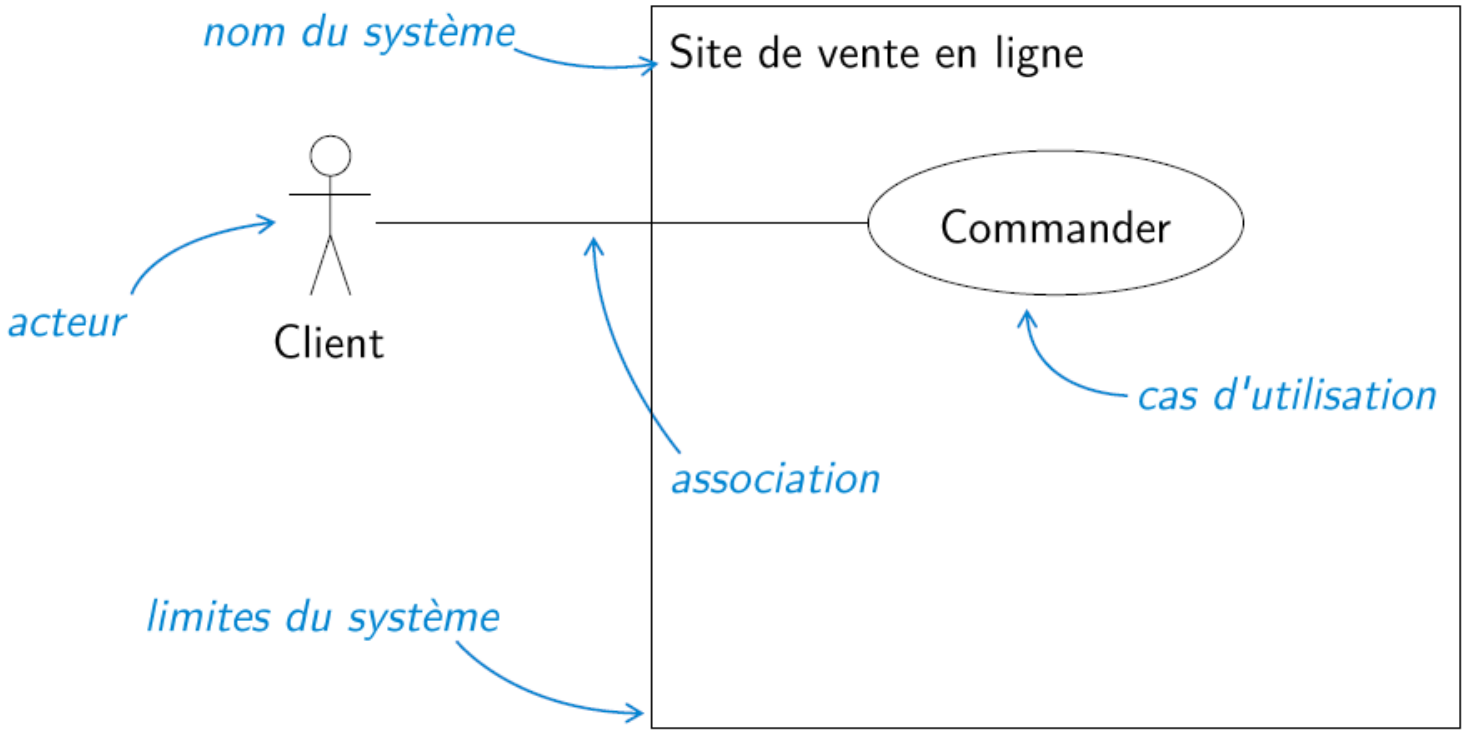
Exemple : site de vente en ligne

Système : Site de vente en ligne

Scénario : Commander

Le client s'authentifie dans le système puis choisit une adresse et un mode de livraison. Le système indique le montant total de sa commande au client. Le client donne ses informations de paiement. La transaction n'est pas autorisée, le système invite le client à changer de mode de paiement. Le client modifie ses informations. La transaction est effectuée et le système en informe le client par e-mail.

Site de vente: Commander



Exemple: cas Weblibrairie

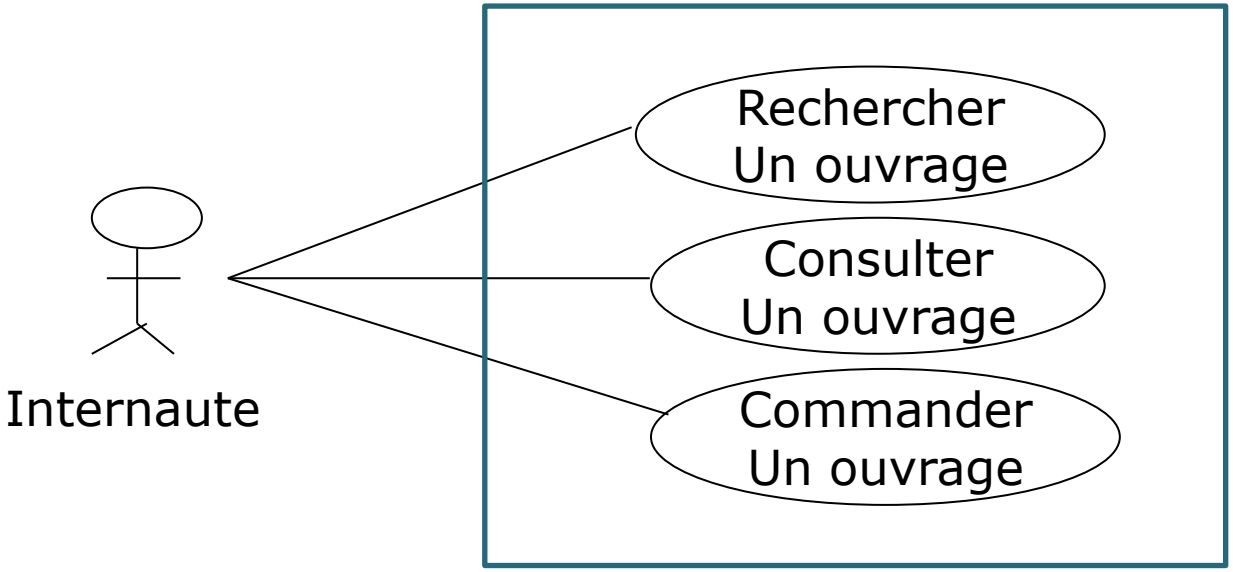
Une librairie veut établir un site de vente des livres en ligne qui doit permettre à l'utilisateur, de :

- Rechercher un livre en fonction de critères de recherche (son titre ou son auteur)
- Consulter un livre (son plan ou sa première page selon les données disponibles dans la base),
- De commander un livre.

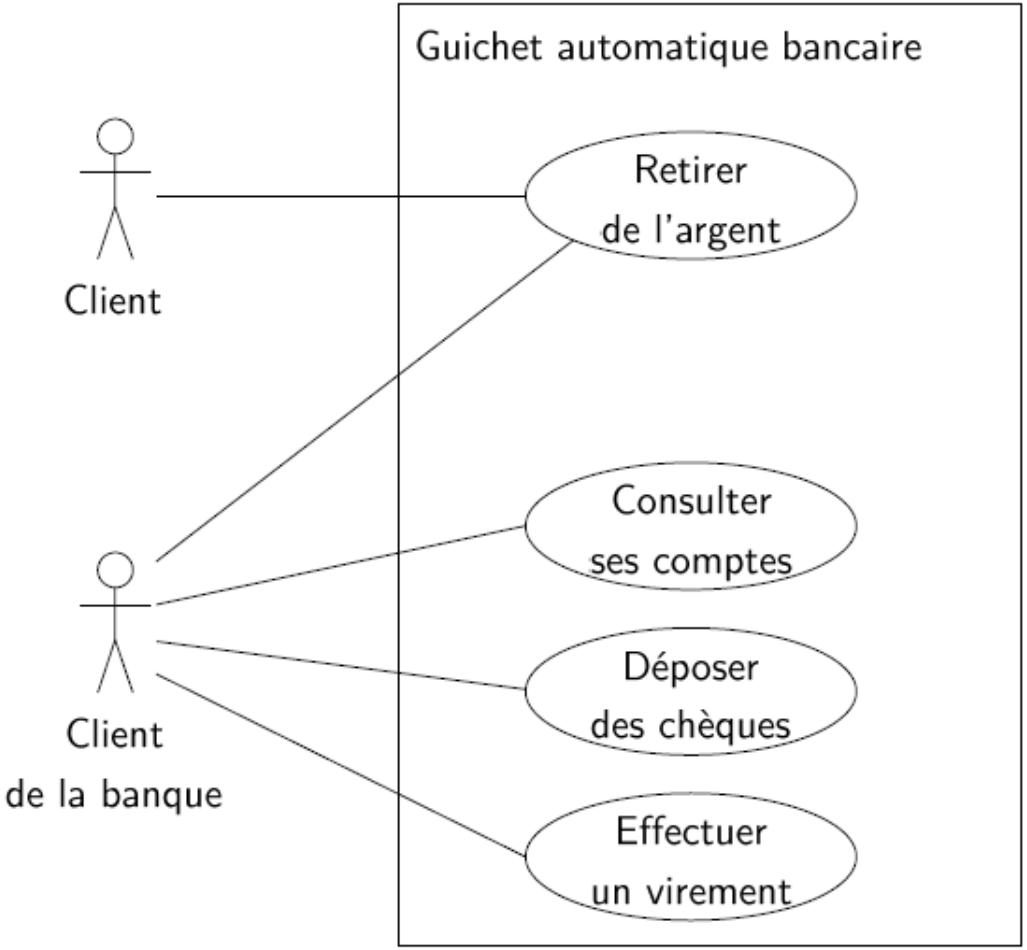
Le service client reçoit les commandes électroniques. De plus, lors de la première connexion, l'internaute doit remplir un formulaire d'identification afin de se connecter en s'identifiant.

Exemple des cas d'utilisation

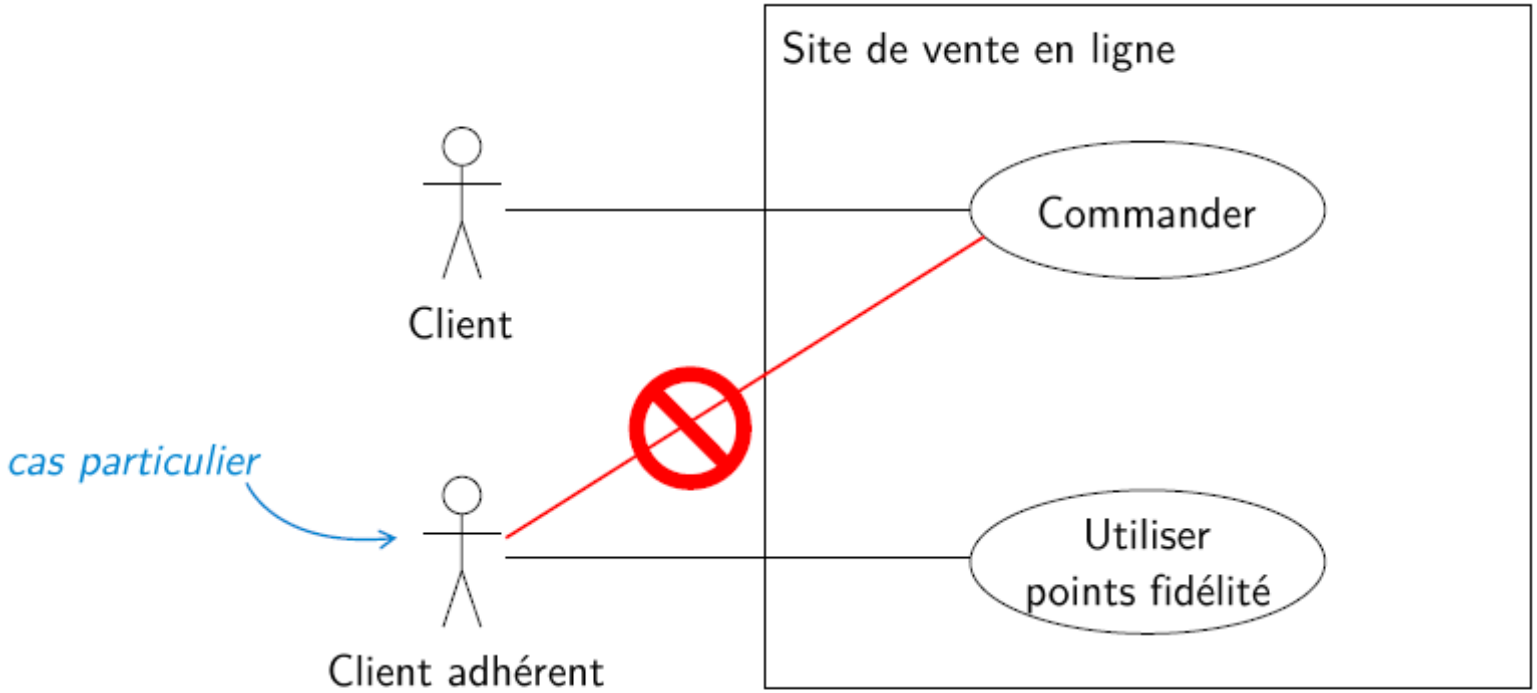
Les cas d'utilisation de WebLibrairie (simplifié) :



Exemple : guichet automatique

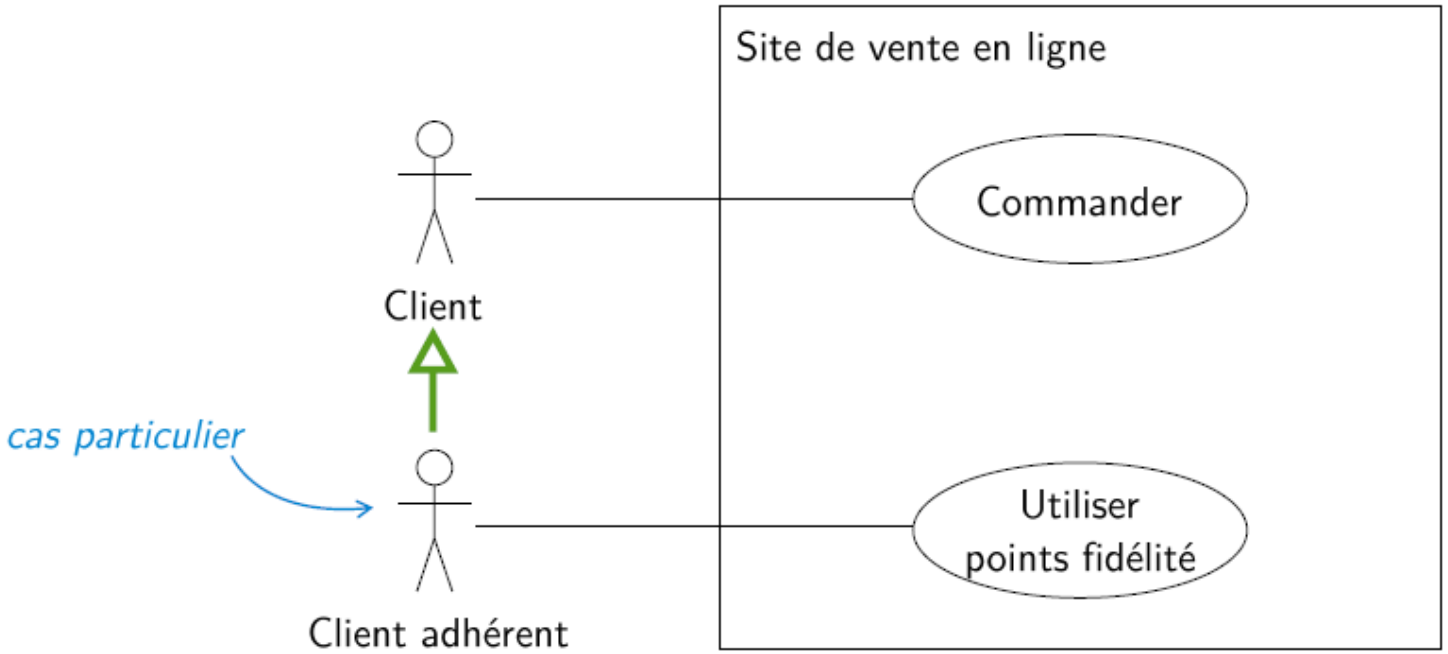


Généralisation de rôle (acteur)



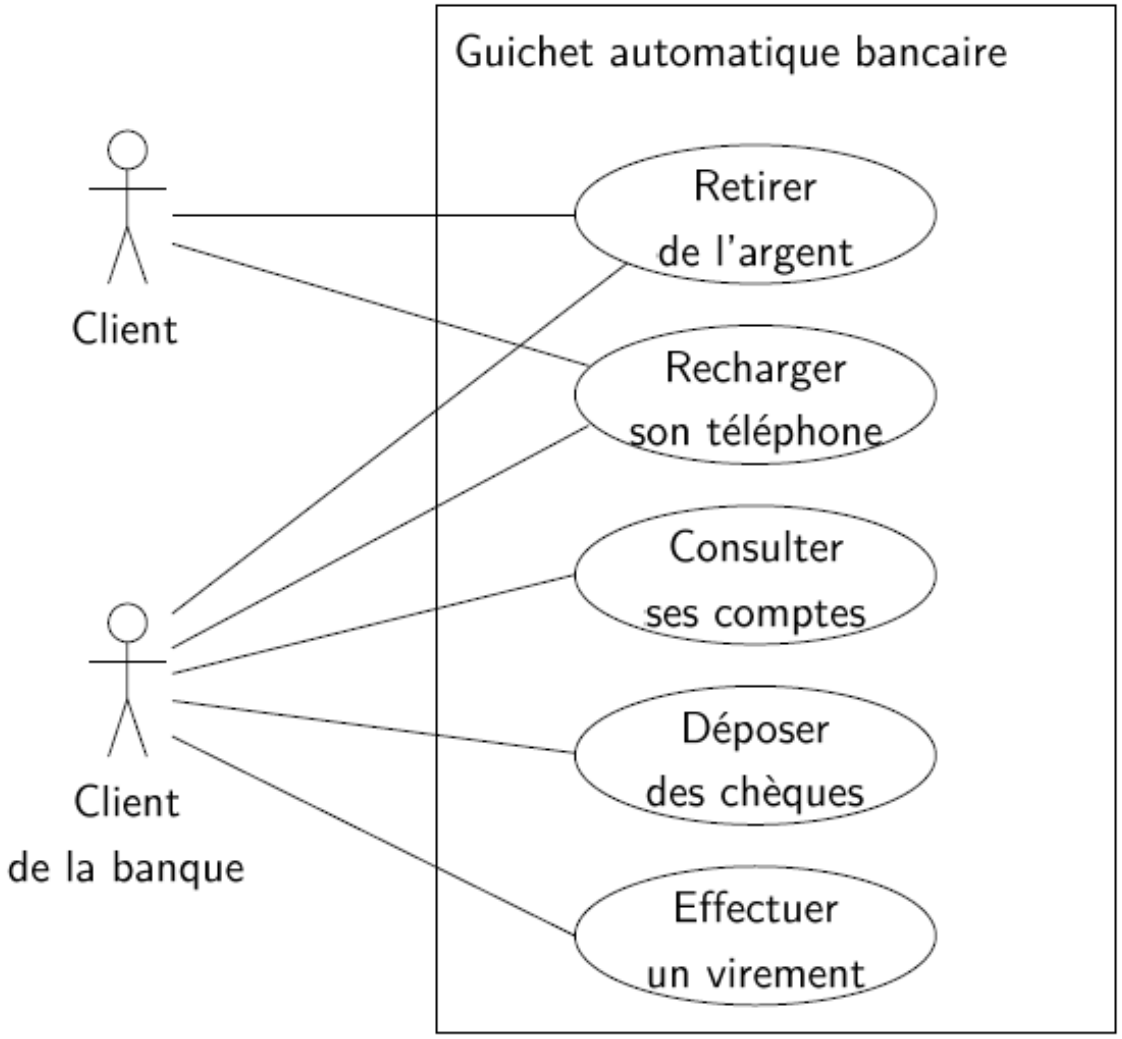
Situation : Y peut faire tout ce que fait X

Généralisation de rôle (2)

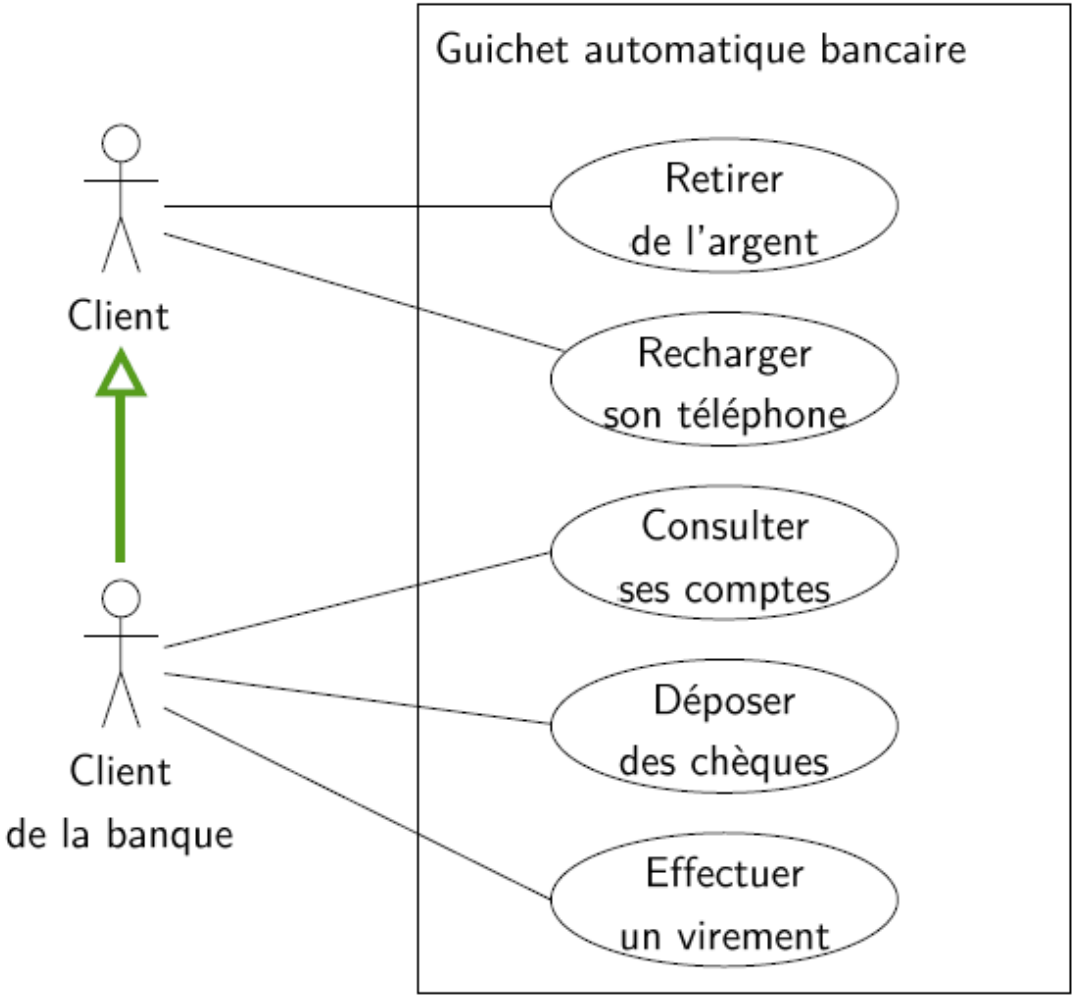


Situation : Y peut faire tout ce que fait X

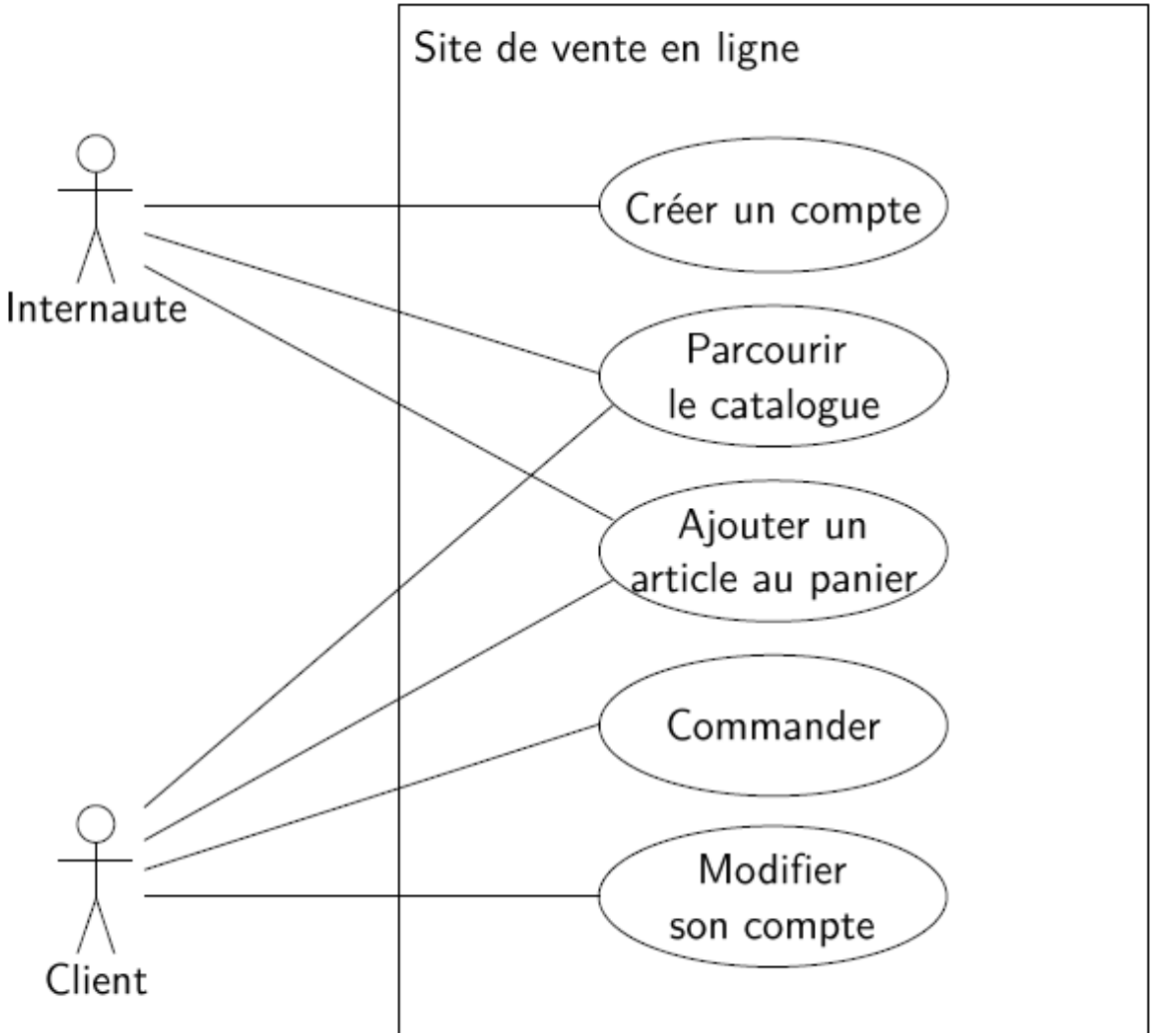
Généralisation de rôle : exemple



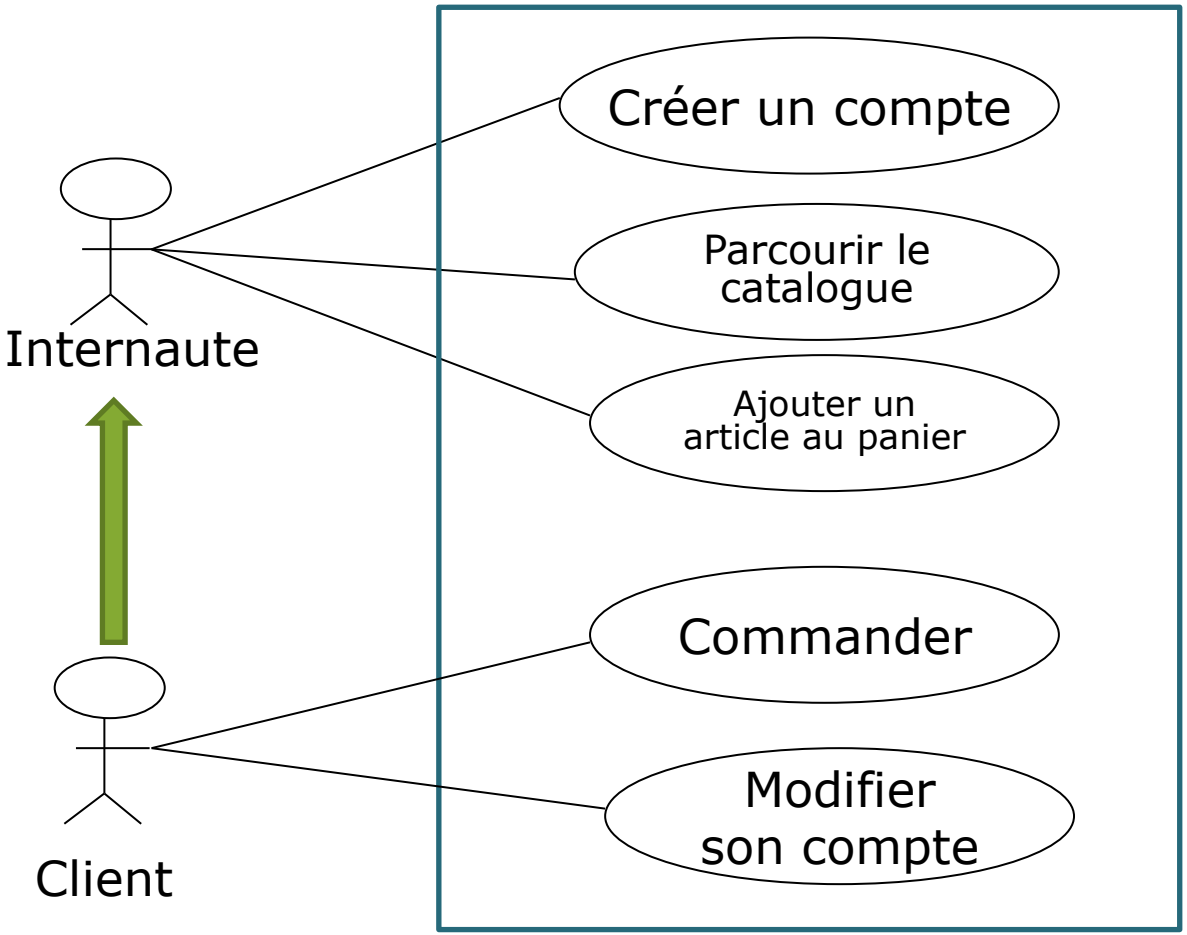
Généralisation de rôle : exemple



Généralisation de rôle : exemple

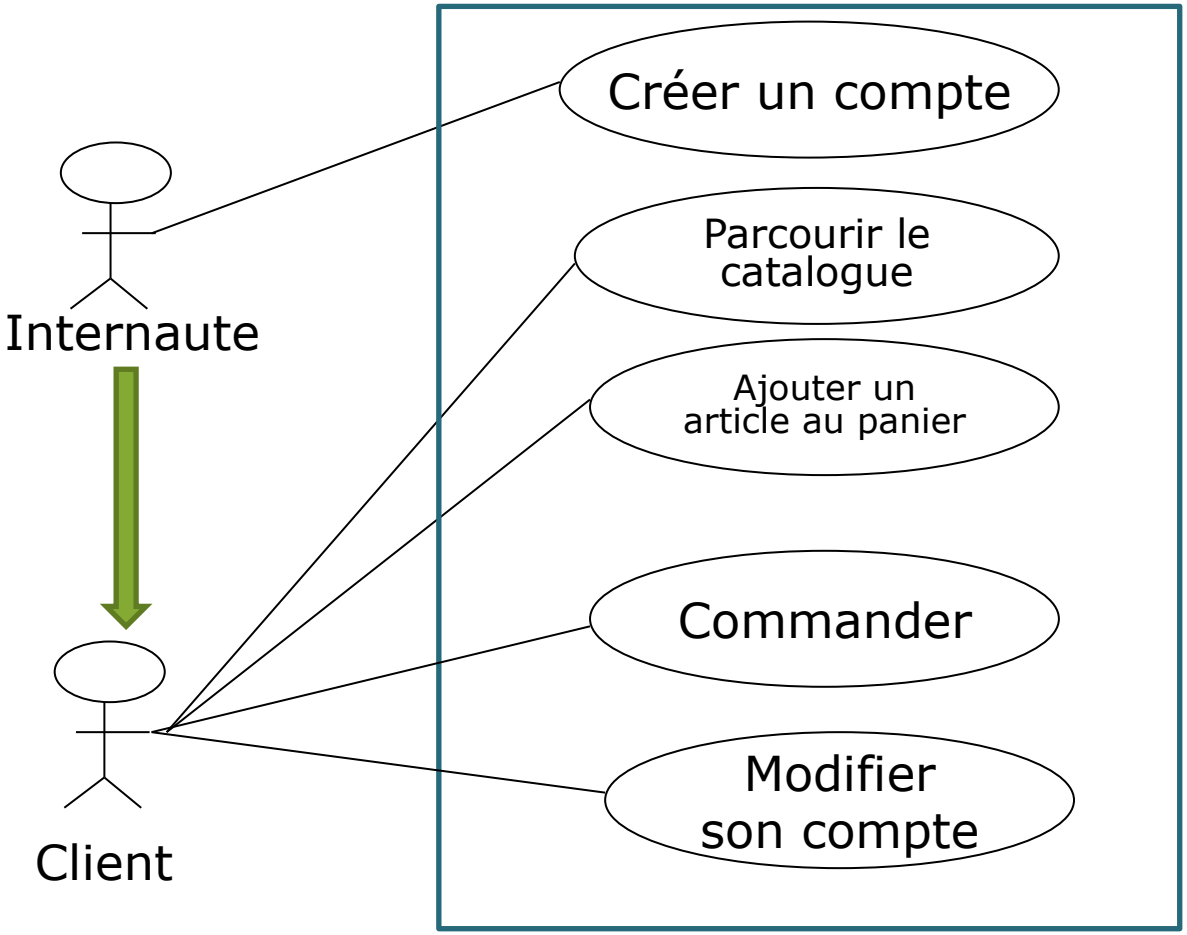


Généralisation de rôle : site de vente



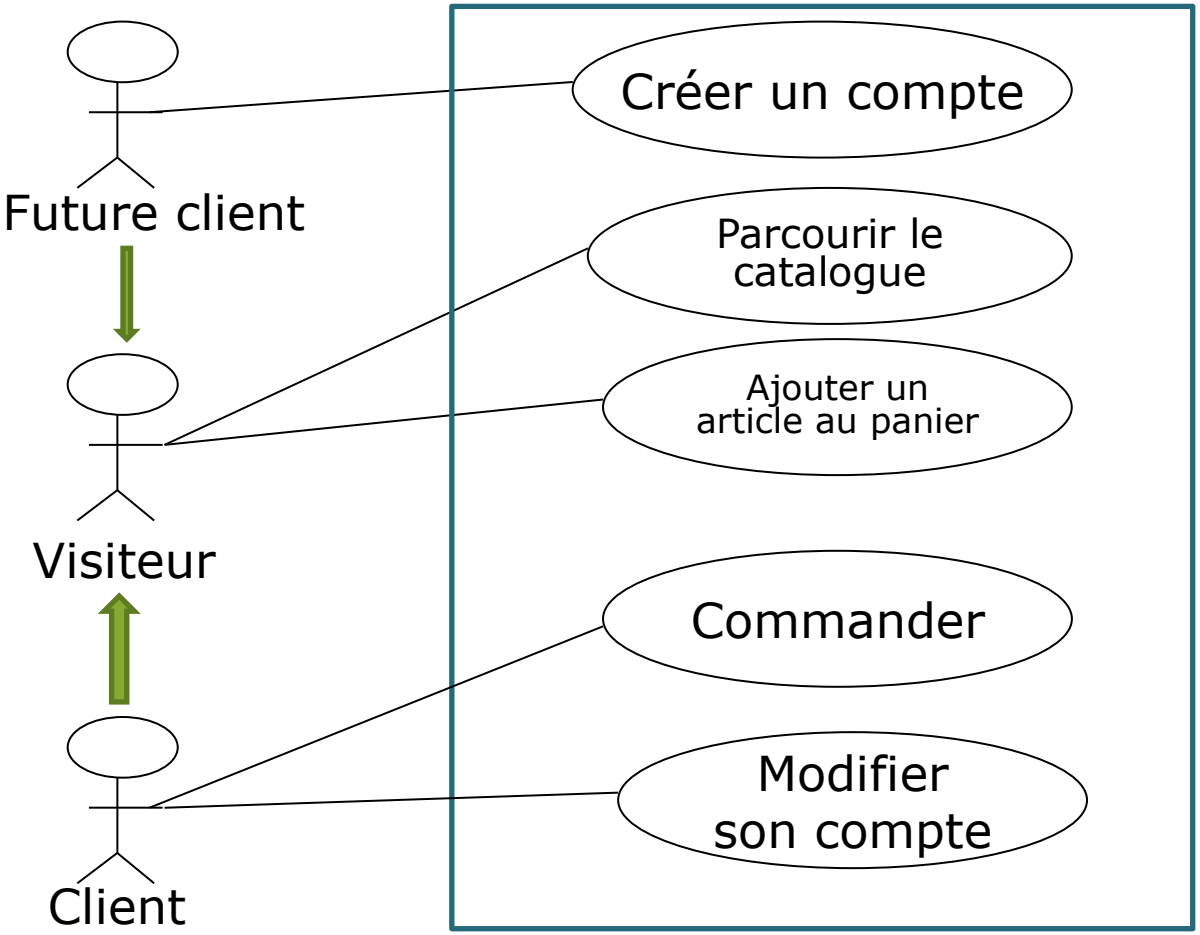
- Est-ce que cette généralisation est –elle possible ?

Généralisation de rôle : site de vente



- Est-ce que celle-ci est –elle possible ?

Généralisation de rôle : site de vente



- Est-ce que celle-ci est possible ?

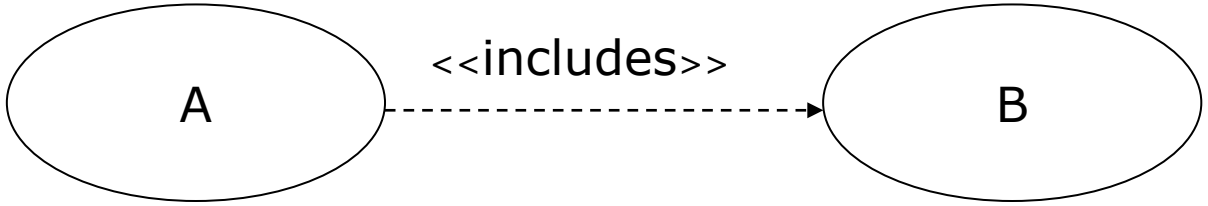
Relations entre cas d'utilisation

UML propose trois types de relations standardisées entre les cas.

- La **relation d'inclusion "inclut"**:

Un cas d'utilisation A comprend obligatoirement le comportement décrit dans le cas d'utilisation B qu'il inclut.

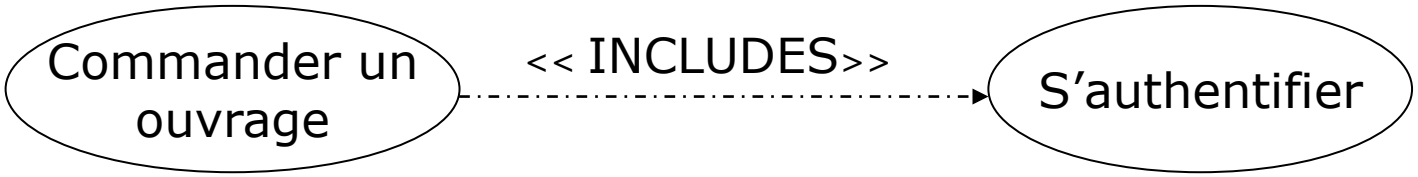
A Inclut B



Relations entre cas d'utilisation (2)

Exemple : Relation d'inclusion entre cas d'utilisation de WebLibrairie :

A INCLUT B

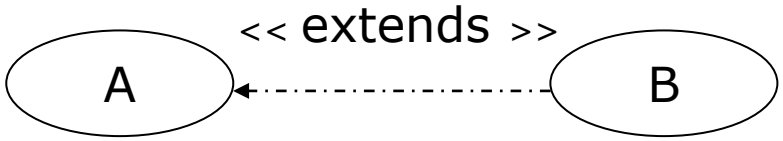


Lorsque l'internaute commande un ouvrage, il est obligatoire qu'il s'authentifie (afin de pouvoir traiter sa commande)

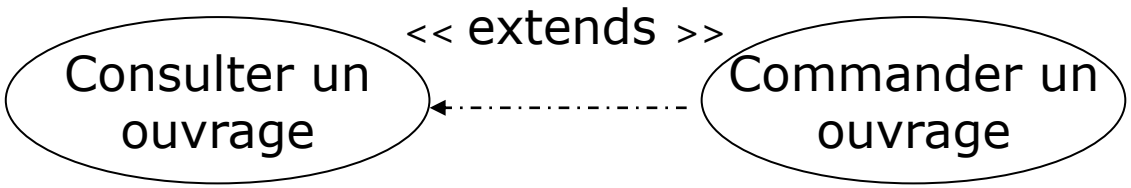
Relations entre cas d'utilisation (3)

- **La relation d'extension "extends"**

Un cas d'utilisation B peut ajouter son comportement au cas d'utilisation A qu'il étend:



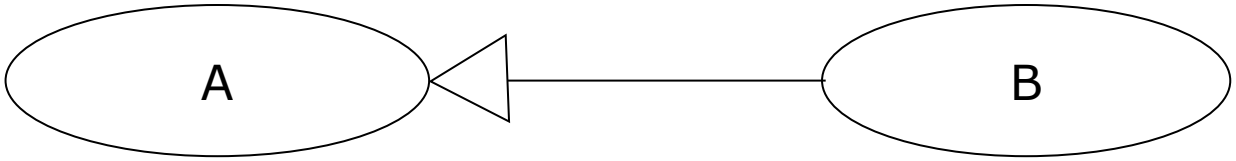
Exemple : Relation d'extension entre cas d'utilisation de WebLibrairie.



Lorsque l'internaute consulte un ouvrage, il peut le commander (ou pas)

La relation d'héritage (Généralisation/Spécialisation)(4)

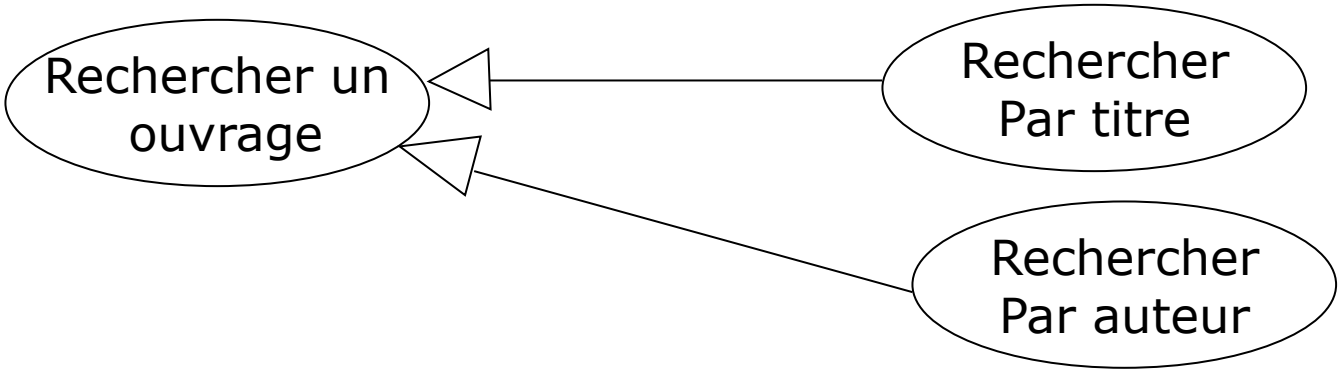
- Un cas d'utilisation descendant hérite de la description de son parent et peut posséder des interactions qui lui sont propres;



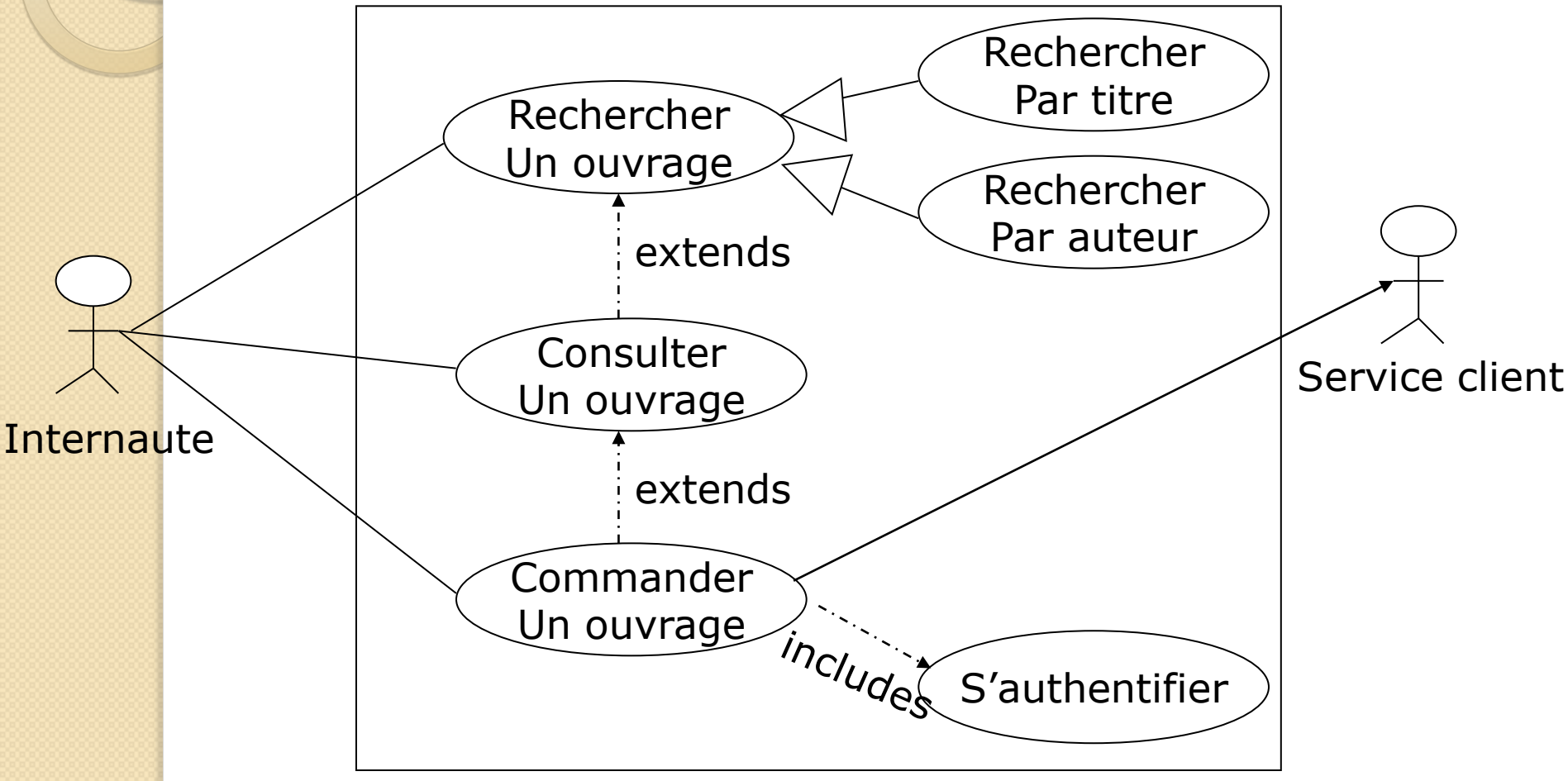
- A généralise B
- B spécialise A
- B hérite des propriétés de A.

Exemple de relation entre cas

- Relation d'héritage entre cas d'utilisation de WebLibrairie



cas d'utilisation de WebLibrairie



Fin du chapitre ……
Description textuelle d'un cas d'utilisation
Diagramme de séquence UML